

Mizan-Tepi University



School of Computing & Informatics

Department of IT

Course Title: Fundamental of database System

Course Code: ITec2071

Project Title: Hospital Management System Project

Group Assignment – Prepared by GROUP 4

2nd Year

1st Semester

Enrolment: Regular (Degree Program)

S/N	Students' Names	ID No.
1.	Mut Gatkek	4493/16
2.	Medeset Fentaye	4429/16
3.	Abaynesh Emiyamreus	3994/16
4.	Mohamed Tahir	4481/16
5.	Alemu Tadele	4704/16
6.	Yohannes Adane	4731/16

Submitted to Instructor: Mrs. Tsige Assefa

Submission date: 08 February, 2025 G.C

 Mizan-Tepi, Ethiopia

Acknowledgement

We, the group members, would like to express our sincere gratitude to [Mrs. Tsige](#), our teacher, for her continuous support, guidance, and encouragement throughout the development of the Hospital Management System project. Her expertise and valuable feedback played a crucial role in the successful completion of this project. We also thank all the other stakeholders and team members who contributed to making this project a success.

We are grateful to our families and friends for their patience and understanding during the course of the project, and we appreciate the resources provided by the university for enabling us to carry out this work.

Table Of Contents

Acknowledgement	2
Introduction	5
Establishing Group Teams	6
Team Table	6
1.1 Planning Phase	8
1.2 Analysis Phase	10
Objectives of the Hospital Management System	10
1.1 figure circle analysis of the Hospital Management System in healthcares	11
Requirement Gathering	12
Feasibility Study	12
ER Diagram for Hospital Management System	16
.....	18
.....	19
Phase 2: Design Phase	20
2.1 Conceptual Design	20
Key Components of Conceptual Design	23
Conceptual Design Notations in Diagrams	25
2.2 Logical Design	27
Entities and Attributes	33
Entities and Attributes	33
Patient	33
Doctor	33
Appointment	34
Billing	34
Pharmacy	35
Relationships	35
2.3 Physical Design	41
1. What Is Physical Design?	41

2. Physical Design Considerations for HMS	41
3. Sample Physical DDL for HMS.....	42
4. Implementation Considerations	46
SQL Table for Patients	47
SQL Table for Doctors.....	47
SQL Table for Appointments	48
SQL Table for Billing	48
SQL Table for Pharmacy	48
Phase 3: Implementation Phase	49
Implementation Considerations	50
Sample DDL and Data Insertion	51
SQL Table for Patients	59
SQL Table for Doctors.....	59
SQL Table for Appointments	60
SQL Table for Billing	60
SQL Table for Pharmacy	60
3. Implementation Considerations	63
Conclusion:.....	64
REFERENCE BOOK:	65

Introduction

This project, which focuses on the development of a Hospital Management System (HMS) , is designed to streamline the operations of a hospital by automating several essential processes such as patient registration, appointment scheduling, doctor management, billing, and pharmacy inventory. The project aims to reduce manual errors, increase operational efficiency, and enhance the overall experience for both hospital staff and patients.

The project is divided into several phases:

Phase 1: Planning and Analysis

The **Planning Phase** involves forming a team of six members, each assigned specific roles based on their expertise. The team includes a Project Manager, System Analyst, Database Designer, SQL Developer, Tester, and Documentation Specialist. Each role is critical to the successful execution of the project. The **Analysis Phase** focuses on gathering requirements, defining the objectives of the Hospital Management System, and conducting a feasibility study to ensure that the project is technically and economically viable.

Phase 2: Design

The **Design Phase** includes the creation of both conceptual and logical designs for the system. The conceptual design defines key modules such as Patient Management, Doctor Management, Billing Management, and Pharmacy Management. These modules structure the system's operations. The logical design establishes the entities and their attributes, such as Patient, Doctor, Appointment, Billing, and Pharmacy, along with their relationships.

Phase 3: Implementation

In the **Implementation Phase**, the database schema is created using SQL to manage all necessary data. The system is tested with sample data, ensuring accurate functionality through SQL queries. Once the system is fully tested, it is deployed in the hospital for real-world use.

.

Phase 1: Planning and Analysis Phase

Establishing Group Teams

The project is developed by a group of six members. Each team member is assigned an ID and a specific role based on their expertise.

Team Table

Team ID	Name	Role	Responsibilities	Phone	Email
4493	MUT	Project Manager	Oversees project planning, progress, and delivery.	0932093 288	mutgatkekdeng@gmail.com
4429	MEDESET	System Analyst	Gathers requirements and ensures they are feasible.	0921789 012	Medeset87@gmail.com
3946	ABAYNESH	Database Designer	Creates the database structure and ensures data relationships.	0989214 532	Abaye73@gmail.com
4481	MOHAMED	SQL Developer	Writes SQL queries and implements database operations.	0917238 971	Tahir17@gmail.com
4704	ALEMU	Tester	Validates the system functionality and fixes errors.	0987345 127	Alex12IT@gmail.com
4731	YOHANS	Documentation Specialist	Prepares all required reports and documentation.	0945132 891	Yohanis34@gmail.com

In the **Planning** of the project, a team of six people is formed, with each person given a specific role based on their expertise. Here down below is explanation of what each team member does:

MUT (Project Manager): MUT is the leader. They make sure everything is planned well, that the project is moving forward, and that it gets completed on time.

MEDESET (System Analyst): MEDESET collects all the requirements for the project. They figure out what needs to be done and make sure it's possible to achieve with the resources available.

ABAYNESH (Database Designer): ABAYNESH designs the database. They decide how the data should be organized and make sure different pieces of data are connected properly.

MOHAMED (SQL Developer): MOHAMED writes the code (SQL queries) to interact with the database. This includes actions like retrieving, inserting, or updating data.

ALEMU (Tester): ALEMU tests the system to make sure everything works correctly. If there are any bugs or errors, they find and fix them.

YOHANS (Documentation Specialist): YOHANS prepares all the reports and documentation needed for the project. This includes writing user guides and any other required documents.

Each team member has a specific job, and by working together, we complete the project successfully.

1.1 Planning Phase

The **Planning Phase** of the Hospital Management System (HMS) project is critical as it sets the foundation for all subsequent phases. This phase involves understanding the needs of the users, defining the system's scope, assessing the available resources, and determining the project's timeline. Here's a detailed explanation of each of these steps:

1. Identifying End-User Requirements

End-users are the people who will interact with the Hospital Management System regularly, including hospital staff (doctors, nurses, receptionists, and administrators) and patients. The process of identifying end-user requirements ensures that the system meets their needs and expectations.

Process:

1. **Interviews and Surveys:** Conduct interviews and surveys with stakeholders such as doctors, nurses, receptionists, and patients to gather input on their needs and preferences.
2. **Workshops:** Organize workshops with hospital staff to discuss the system's functionality and ensure that all necessary features are considered.
3. **Observation:** Observe how the hospital staff currently performs tasks like patient registration, appointment scheduling, and billing. This will provide insights into existing pain points and areas for improvement.
4. **Prototyping:** Develop wire-frames or prototypes of the system to give stakeholders an idea of how the system might function and gather feedback for adjustments.

End-user Requirements for the Hospital Management System might include:

- **Patient Management:** Patients should be able to register, update personal information, and view their medical records.
- **Appointment Scheduling:** Users (patients and doctors) should be able to book, modify, and cancel appointments.
- **Billing:** The system should track billing, payments, and generate invoices.
- **Pharmacy Management:** Track the availability of medications and assist in ordering restocks.
- **Data Security:** Ensure the protection of patient data against unauthorized access.

1 Methods for Requirement Collection:

- **Interviews:** Talk to doctors, nurses, and hospital administrators.

- **Surveys/Questionnaires:** Distribute surveys to collect structured feedback from users.
- **Focus Groups:** Gather a small group of end-users to discuss the system's functionality and gather more detailed insights.
- **Prototyping:** Show end-users mock-ups or wireframes to help them visualize the system.

2. Determining the Scope and Objectives

Defining the **scope** and **objectives** of the project helps ensure that the system's goals align with the hospital's needs, resources, and timeline.

Scope Definition:

The scope defines the boundaries of the Hospital Management System project. It outlines what is included in the project and what is excluded, which helps to avoid scope creep (uncontrolled changes or continuous growth in a project's scope).

For example:

In Scope:

- Patient registration
- Doctor appointment scheduling
- Billing and payment processing
- Pharmacy stock management
- Staff management (doctors, nurses, administrators)
- Reports generation for hospital management

Out of Scope:

- Advanced medical equipment management
- Integration with external hospital systems
- Insurance claim processing

This Hospital Management System Database Design has an **ER Diagram for Hospital Management System project**.

This material includes **database schema, Data dictionaries, Database Tables and ERD**.

1.2 Analysis Phase

Objectives of the Hospital Management System

1. Automate patient registration and medical record management.
2. Enable efficient scheduling of appointments and staff management.
3. Manage billing, payments, and pharmacy stock.
4. Ensure data integrity, security, and user-friendly interaction.

Here's explanation of the objectives of a **Hospital Management System (HMS)**:

Automate patient registration and medical record management:

1. This means that when a patient visits a hospital, their personal and medical details are entered into a system automatically, which helps in creating and managing their medical records without using paper. This ensures that patient information is easily accessible and up-to-date

Enable efficient scheduling of appointments and staff management:

1. The system helps in scheduling patient appointments with doctors, ensuring that there is no overlap. It also helps in managing staff schedules, such as when doctors or nurses are available, improving the hospital's overall organization.

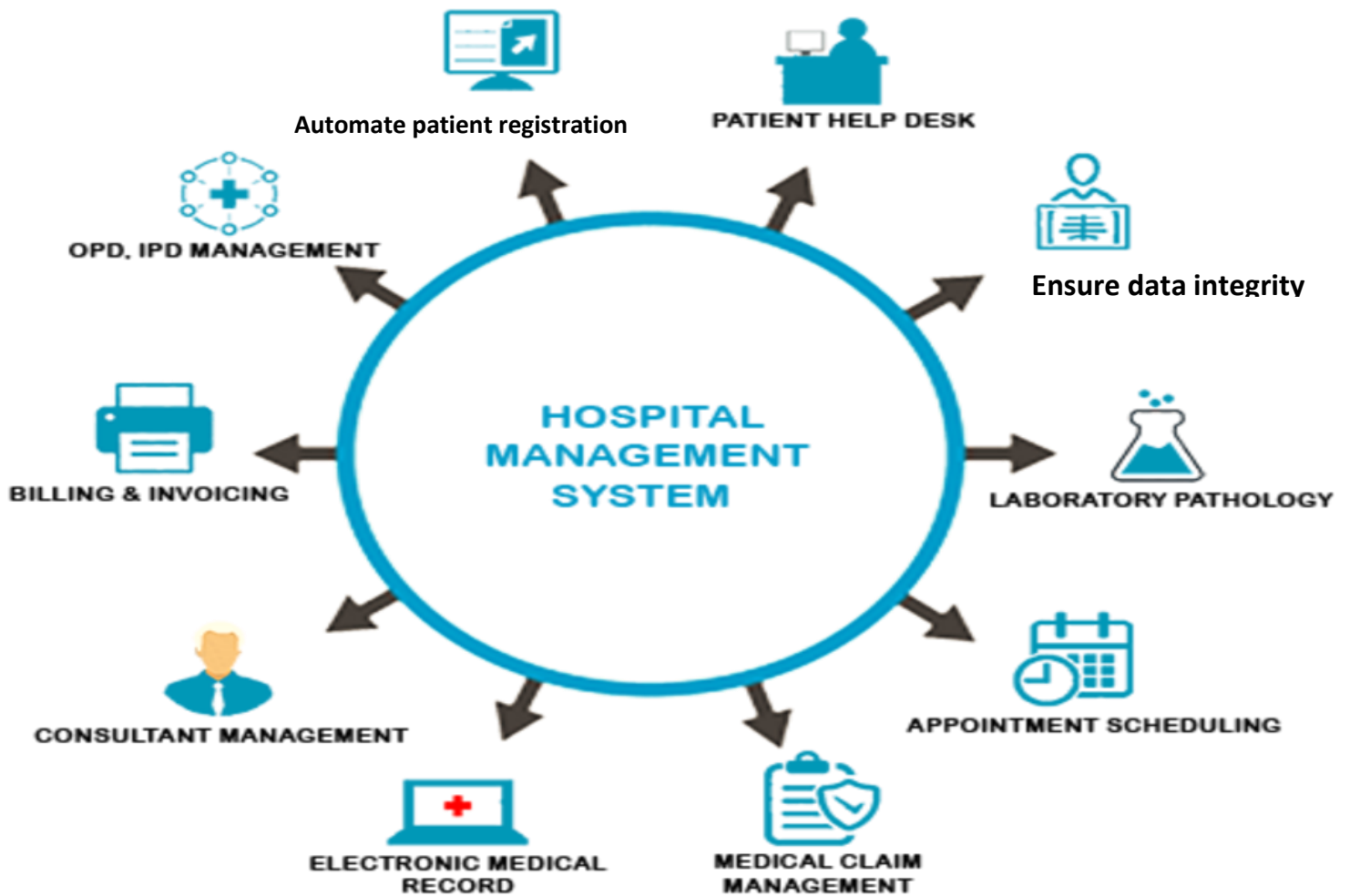
Manage billing, payments, and pharmacy stock:

1. The system helps track patient bills, payments made, and manages the stock of medicines in the pharmacy. It ensures that billing is accurate and payments are recorded properly. It also tracks the availability of medicines, so the pharmacy doesn't run out of essential items.

Ensure data integrity, security, and user-friendly interaction:

1. The system ensures that all the data entered into it is accurate, up-to-date, and protected from unauthorized access. It is designed to be easy for hospital staff to use, so they can focus more on helping patients and not struggle with complicated software.

In short, an HMS helps the hospital run more smoothly by organizing patient data, appointments, billing, and pharmacy stock while keeping everything secure and easy to use.



1.1 figure circle analysis of the Hospital Management System in healthcares

The diagram above illustrates the objectives of a Hospital Management System. It visually breaks down each objective using icons, making it easy to understand how the system operates across different functional areas.

Requirement Gathering

Requirement Type	Description	Source
Functional	Allow patients to book and view appointments online.	Patients
Functional	Manage doctor schedules and availability.	Hospital Admin
Non-functional	Ensure the system responds to user requests in under 3 seconds.	IT Team
Technical	Use Xampp, MySQL database to store and retrieve data.	Developers

Feasibility Study

- 1. Technical Feasibility:** SQL is compatible with hospital IT infrastructure.
- 2. Economic Feasibility:** Open-source tools reduce development costs.
- 3. Operational Feasibility:** Streamlines existing hospital operations.

Technical Feasibility

- Explanation:** The system is technically feasible as **SQL** is compatible with the hospital's existing IT infrastructure. The hospital's current IT setup can support

SQL, and developers are familiar with this database management system, making it a good choice.

Economic Feasibility

- **Explanation:** The use of **open-source tools** for development helps reduce costs. Open-source technologies like MySQL (for databases), PHP (for backend development), and JavaScript (for frontend development) reduce licensing fees and overall development costs.

Operational Feasibility

- **Explanation:** The system will streamline existing hospital operations. It will help automate processes like patient registration, appointment scheduling, billing, and stock management, thereby improving operational efficiency and reducing manual errors.

4. Key Activities in the Analysis Phase

The **Analysis Phase** includes several key activities that help in understanding the hospital's needs, gathering system requirements, and designing a data model.

Activity 1: Identify the Problem

- **Explanation:** The key problem addressed by the HMS is the lack of automation in the hospital, leading to inefficiency, errors in billing and record-keeping, and difficulties in managing appointments and staff schedules. The system aims to automate and streamline these processes.

Activity 2: Gather Requirements

- **Explanation:** This step involves collecting input from various stakeholders, including doctors, nurses, hospital administrators, IT staff, and patients. This ensures that all aspects of hospital management are considered. The requirements gathered include both functional (e.g., appointment booking, billing) and non-functional (e.g., system performance, security) aspects.

Activity 3: Identify Data Entities

- **Explanation:** **Data entities** represent the main objects of the system that need to be stored and managed. These entities will be used to build the database schema.

- **Example Data Entities:**

- **Patient:** Includes attributes such as PatientID, Name, Age, Gender, Address, etc.
- **Doctor:** Includes attributes like DoctorID, Name, Specialty, Availability.
- **Appointment:** Includes AppointmentID, Date, Time, PatientID, DoctorID.

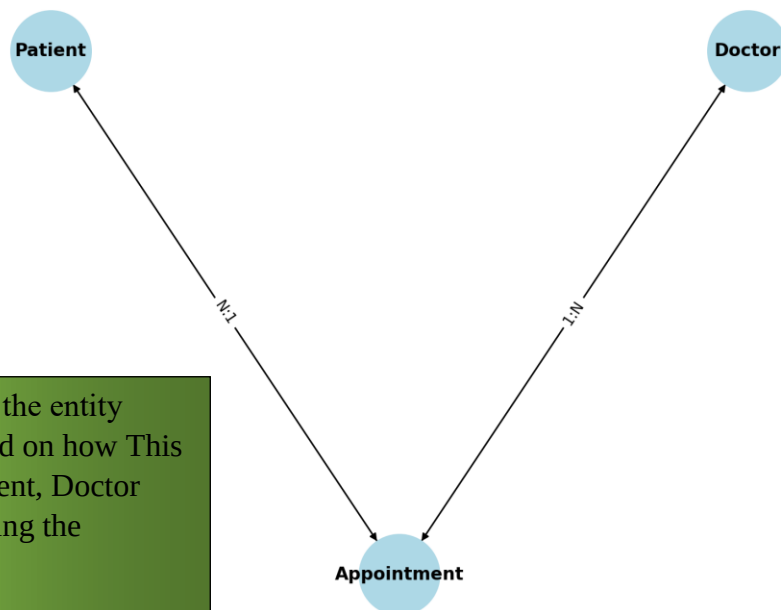
Activity 4: Define Attributes

- **Explanation: Attributes** are characteristics or properties of the data entities. For instance:
 - The **Patient** entity may have attributes like PatientID, Name, ContactInfo, etc.
 - The **Appointment** entity may have attributes like Date, Time, and links to the **Patient** and **Doctor** entities.

Activity 5: Identify Relationships

- **Explanation: Relationships** define how data entities are associated with one another. For instance:

Entity Relationship Diagram (ERD)



Here's an example of ERD , it is the entity relationship diagram (ERD) based on how This example shows relationship patient, Doctor and Appointments to understanding the database design better .

1.2.figure for simple ER diagram r/ships

- A **Patient** can have many **Appointments** (1:N relationship).
- A **Doctor** can have many **Appointments** (1:N relationship).
- An **Appointment** links both a **Doctor** and a **Patient** (N:1 relationship).

Activity 6: Create Data Flow Diagrams (DFDs)

- **Explanation:** A **Data Flow Diagram (DFD)** is a visual representation that shows how data moves within the system. It depicts processes, data stores, and external entities. In the context of the HMS, the DFD may show processes like **Patient Registration**, **Appointment Scheduling**, and **Billing**. It will also show data flow between these processes.

Activity 7: Develop Entity-Relationship (ER) Diagram

- **Explanation:** An **Entity-Relationship Diagram (ERD)** shows how entities are related to one another. It illustrates the relationships, attributes, and primary keys of each entity.

What is Hospital Management System Database Design?

The ***Hospital management system database design*** is a database design use for managing **hospital** functions and events. It enables the admin to register a patient for the hospital, stores their disease details into the database. Any of the staff members, doctor & admin is able to add, view, edit, update or delete data.

What is the purpose of Hospital management System database?

The purpose of the [Hospital Management System](#) database Design is to make a secure and easy way of storing information of the patient, doctors, inpatient, outpatient, Rooms, and Bill payment.

What are the Features of the Hospital Management System Database Design?

There are seven(7) common features of Hospital Management System Database Design such as Managing Patients, Doctors, laboratory, Inpatient, Outpatient, Rooms, and Hospital Bills information.

ER Diagram for Hospital Management System

ER Diagram Hospital Management System Database Design Project.

- **Manage Patients**

This table of ER Diagram Hospital Management System will be used for storing and managing the patient information.

- **Manage Doctor**

This table will be used for storing and managing the Doctor information and login account.

- **Manage Laboratory**

This table will be used for storing and managing the Laboratory transaction.

- **Manage Inpatient**

This table will be used for storing and managing the inpatient information and diagnosis.

- **Manage Outpatient**

This table will be used for storing and managing the inpatient information and diagnosis.

- **Manage Room**

This table will be used for storing and managing the room information and assigning of patient in every room.

- **Manage Bills**

The billing table will be used for managing the statement of accounts per patient and for the collection of bills.

ER Diagram for Hospital Management System with Relationship

This section will show you the **ER Diagram for Hospital Management System Project**.

By following this ER diagram for HMS, you will gain a better understanding of how to create an ER diagram for any database design or backend development.

Below on the next page, you will see a diagram/image of the ER diagram for the Hospital Management System, which visually represents the relationships between different entities in the system

In the image below you see the actual relationship of all entity.

Diagram of Hospital Management System

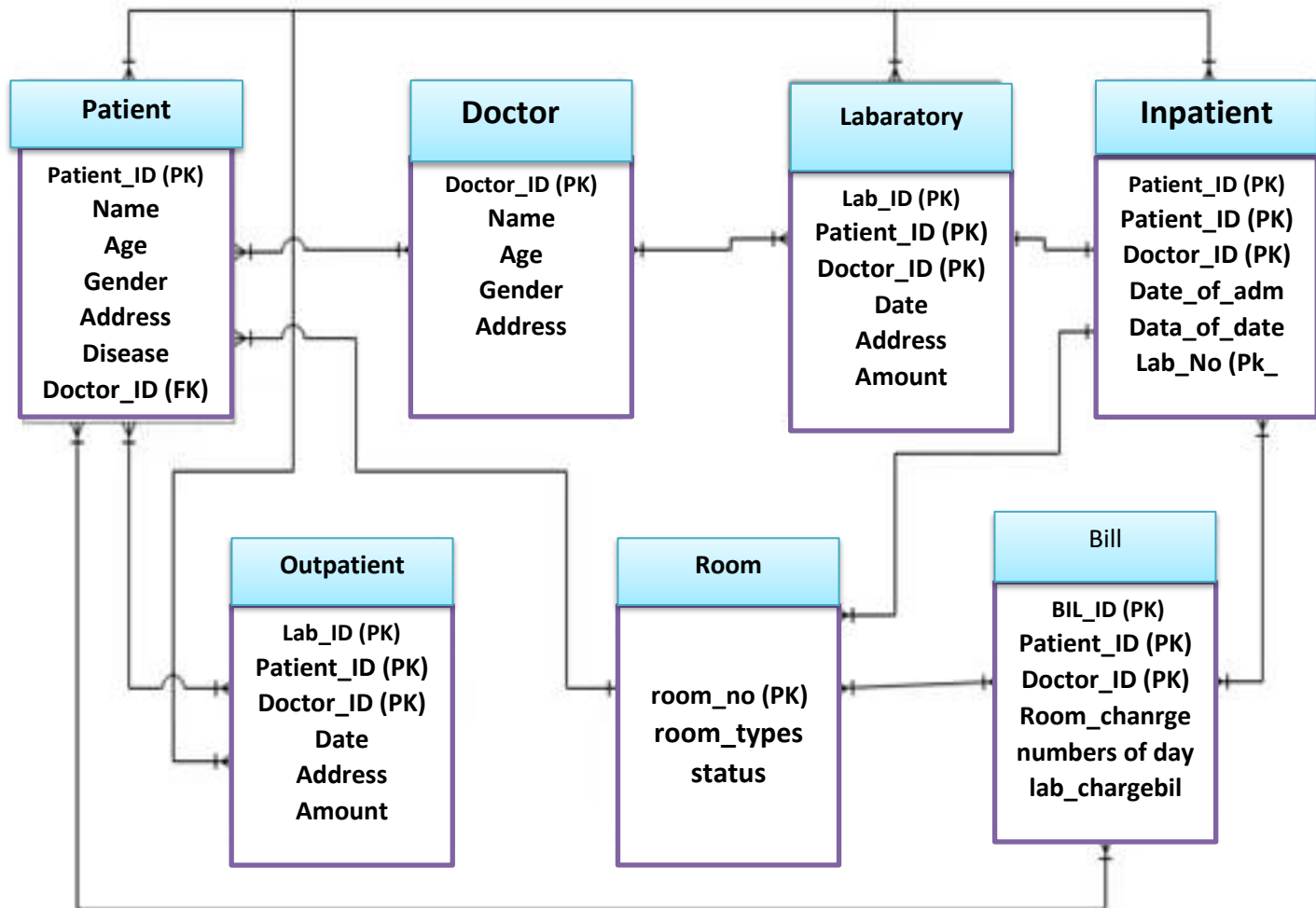
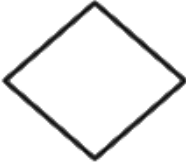


Figure 1.3 Hospital Management System Entity Relationship Diagram

Based on figure 1.3, the proposed **ER diagram for hospital management system with relationship** shows the entity of the proposed SCA Hospital Management system database, which is presented by tables; the tables are made to meet the required specification of the system and provide much more specific details of each entity within the system.

Notation ER Diagrams for Hospital Management System (HMS).

Symbols	Meaning
	Entity
	Relationship
	Attribute
	Key Attribute
	Cardinality Ratio N: 1 for E1: E2 in R

1.4. Notation ER Diagrams, there's many Notation ER Diagrams, as we know but we show that as a sign of the ER-notation diagram to better understand, about the of Entity-Relationship Diagrams (ERD).

Phase 2: Design Phase

2.1 Conceptual Design

The conceptual design defines the main modules of the system:

Conceptual database design: is the process of identifying the essential data elements, relationships, and constraints in a data model. In conceptual database design the important activities are to identify

- **Entity**
- **Attribute**
- **Relationship**
- **Constraints**

1. Entity:

- An **entity** represents a real-world object or concept that can be distinctly identified in a system. In database systems, an entity is often a table, and each row in the table represents a specific instance of the entity.
- An **entity** has a set of attributes that describe it and its properties.
- For example:
 - In a **Hospital Management System (HMS)**, entities could include Patient, Doctor, Room, Laboratory, Bill, etc.
 - These entities hold data related to patients, doctors, rooms, and bills, respectively.

Example:

- **Entity:** Patient
- **Attributes:** id, first_name, last_name, contact_info, insurance_details, etc.

2. Relationship:

- A **relationship** defines the connection between two or more entities. Relationships in a database describe how entities are related to each other and help establish associations.
- Relationships can have attributes of their own, known as **relationship attributes**.

Relationships can be categorized as:

- **One-to-One:** One instance of Entity A is related to one instance of Entity B.
- **One-to-Many:** One instance of Entity A is related to many instances of Entity B.
- **Many-to-Many:** Many instances of Entity A are related to many instances of Entity B.

Example:

- **Entities:** Patient, Doctor
- **Relationship:** Appointment (A Patient has an Appointment with a Doctor).

3. Attributes:

- **Attributes** are characteristics or properties that describe an entity. They are the data that is stored for each entity.
- Each attribute holds a value or a set of values. Attributes can be:
 - **Simple:** A single value (e.g., first_name, last_name).
 - **Composite:** An attribute that can be divided into smaller sub-parts (e.g., address can be divided into street, city, zipcode).
 - **Derived:** Attributes that can be calculated from other attributes (e.g., age can be derived from date_of_birth).
 - **Multi-valued:** An attribute that can hold multiple values (e.g., a patient may have multiple phone_numbers).
- **Examples:**
 - For the Patient entity:
 - first_name, last_name, contact_info, insurance_details, address
 - For the Doctor entity:
 - first_name, last_name, specialty, contact_info

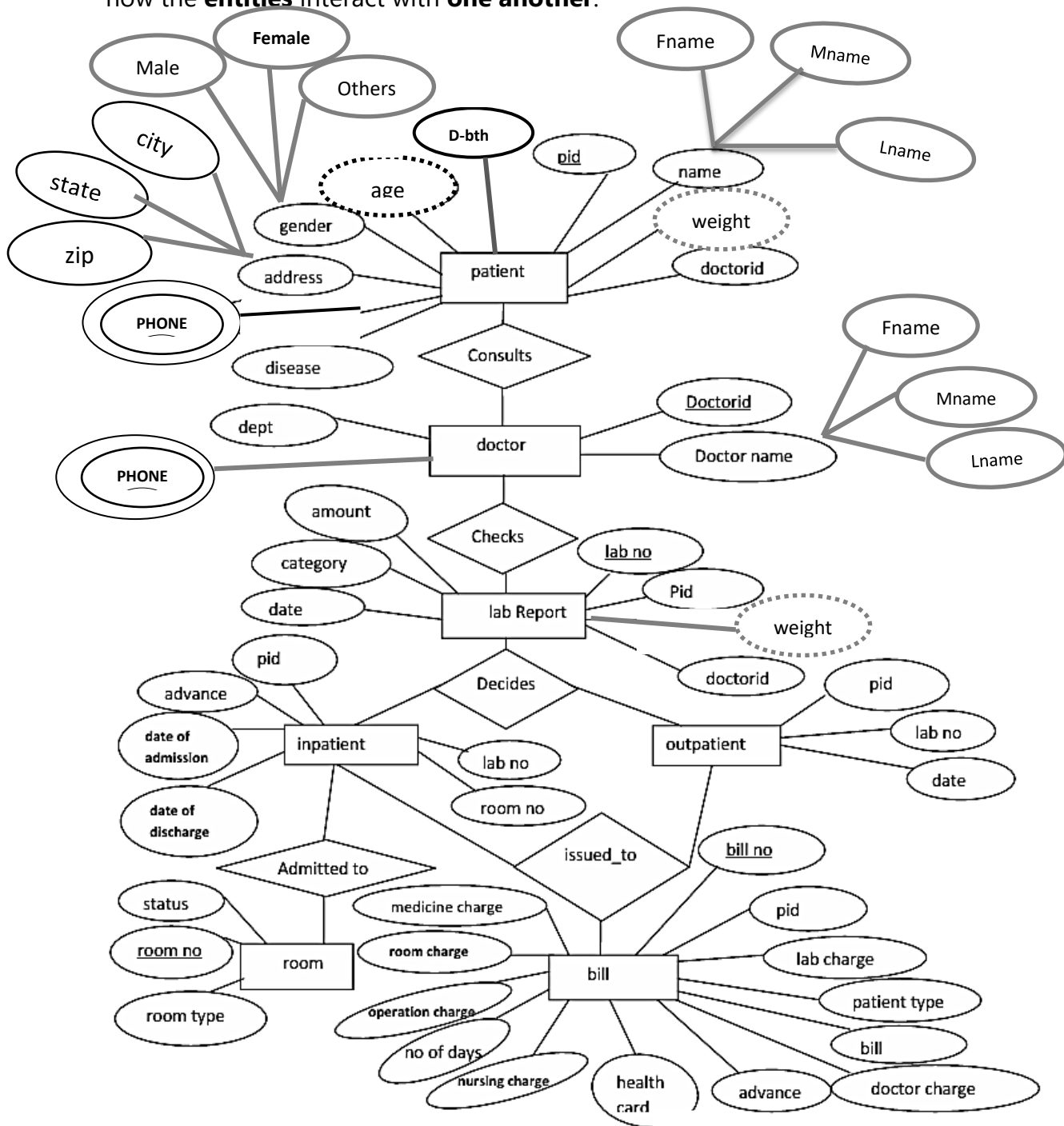
4. Constraints

- **Constraints** define rules for the database to ensure **data integrity** and **accuracy**.

Types of Constraints

- **Primary Key (PK)** – A unique identifier for each record in an entity. (e.g., patient_id in the Patient table).
- **Foreign Key (FK)** – Establishes a relationship between two entities by referencing the primary key from another table. (e.g., doctor_id in the Appointments table refers to doctor_id in the Doctor table).
- **Unique Constraint** – Ensures that no duplicate values exist in a column. (e.g., Patient's contact number must be unique).
- **Not Null Constraint** – Ensures that a column cannot have a NULL value. (e.g., A patient must have a first name).

Let us look at the diagram to better understand the **conceptual design** with notations, so you'll have a clear understanding of how the system's database will be structured and how the **entities** interact with **one another**.



1.5 figure Hospital Management System (HMS) structured and how the **entities** interact with **one another**.

Explanation of Conceptual Design

As we know Conceptual design is the **first step** in designing a database system. It provides a **high-level overview** of how the system's data will be organized and how different entities (objects) interact with each other. This stage focuses on **understanding the business requirements** and structuring the data accordingly, without worrying about implementation details like database types, indexing, or storage.

Key Components of Conceptual Design

1. Entities

An **entity** represents a real-world object or concept about which we store data in the system. Each entity is associated with certain **attributes** that describe its properties.

Example Entities in a Hospital Management System (HMS)

- **Patient** (Stores patient details)
- **Doctor** (Stores doctor information)
- **Appointment** (Records scheduled patient visits)
- **Billing** (Manages patient payments)
- **Room** (Tracks patient accommodations)
- **Laboratory** (Stores test results)

2. Attributes

Attributes define **characteristics or properties** of an entity. Each entity has a set of attributes that help describe it.

Types of Attributes

- **Simple Attributes** – Contain a single value (e.g., first_name, age).
- **Composite Attributes** – Can be divided into smaller attributes (e.g., address → street, city, zipcode).
- **Derived Attributes** – Can be calculated from other attributes (e.g., age derived from date_of_birth).
- **Multi-Valued Attributes** – Can have multiple values (e.g., a patient can have multiple phone numbers).

Example of Entity and Attributes

Patient Entity Attributes:

- patient_id (Primary Key)
- first_name
- last_name
- date_of_birth
- contact_info
- insurance_details

3. Relationships

A **relationship** defines how two or more entities are connected. It helps to **show interactions** between entities in the system.

Types of Relationships

1. **One-to-One (1:1)** – A single entity instance is associated with one instance of another entity.
 - Example: A **patient** is assigned to **one room** in the hospital.
2. **One-to-Many (1:M)** – One entity instance is related to multiple instances of another entity.
 - Example: A **doctor** can treat **many patients**, but each patient is treated by **one doctor** at a time.
3. **Many-to-Many (M:N)** – Multiple instances of one entity can be related to multiple instances of another entity.
 - Example: A **patient** can have multiple **lab tests**, and each **lab test** can be performed for multiple **patients**.

Example Relationships

- **Patient** → **Appointments** → (One patient can have many appointments, but an appointment is linked to one patient).
- **Doctor** → **Specialties** → (One doctor can have multiple specialties, and multiple doctors can share the same specialty).
- **Billing** → **Payments** → (A patient can have multiple payments for different services).

4. Constraints

Constraints define rules for the database to ensure **data integrity** and **accuracy**.

Types of Constraints

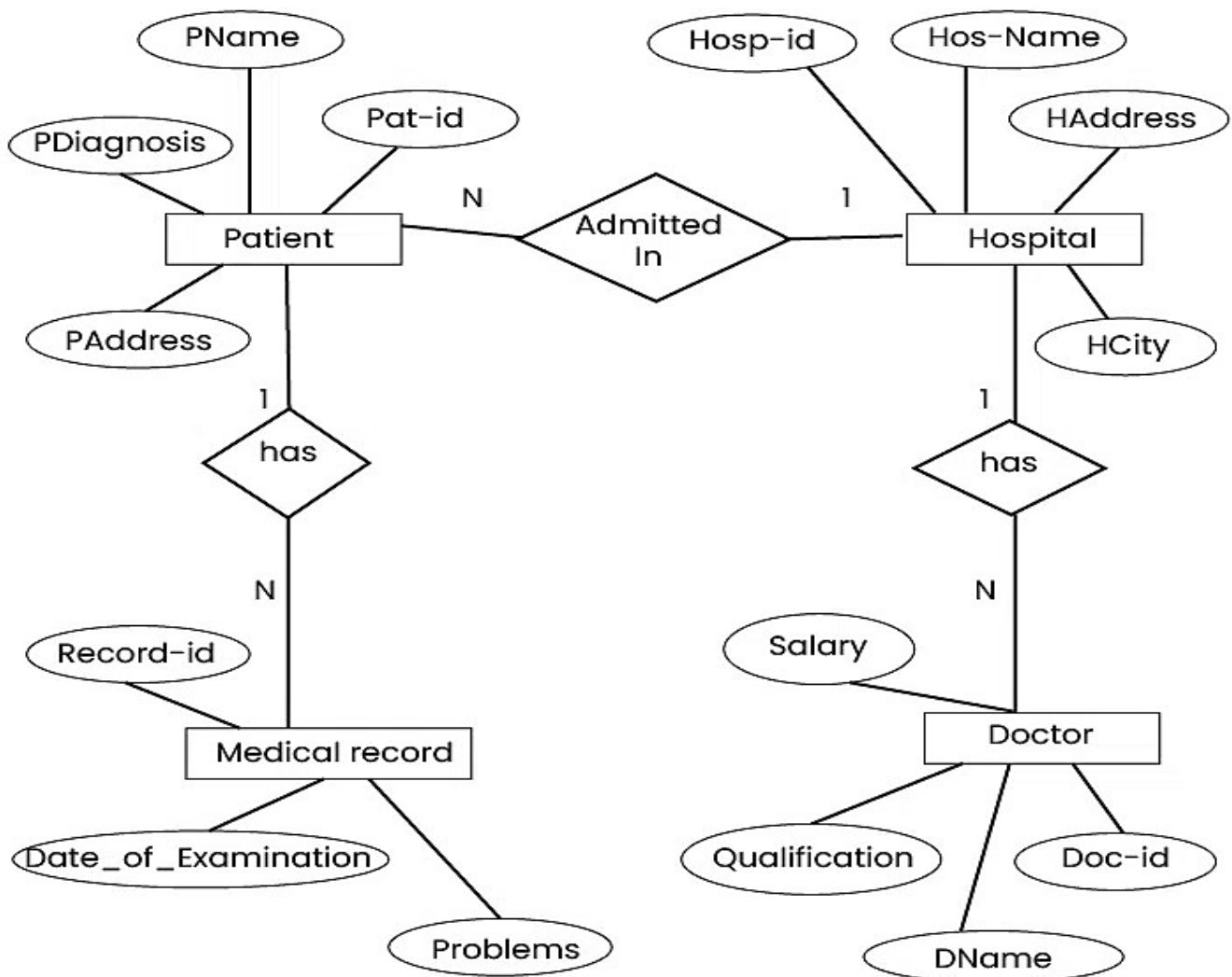
- **Primary Key (PK)** – A unique identifier for each record in an entity. (e.g., patient_id in the Patient table).
- **Foreign Key (FK)** – Establishes a relationship between two entities by referencing the primary key from another table. (e.g., doctor_id in the Appointments table refers to doctor_id in the Doctor table).
- **Unique Constraint** – Ensures that no duplicate values exist in a column. (e.g., Patient's contact number must be unique).
- **Not Null Constraint** – Ensures that a column cannot have a NULL value. (e.g., A patient must have a first name).

Conceptual Design Notations in Diagrams

In **Entity-Relationship Diagrams (ERD)**, we use **notations** to represent the conceptual design visually:

- **Rectangles** → Represent **entities** (e.g., **Patient**, **Doctor**).
- **Ellipses** → Represent **attributes** (e.g., name, age).
- **Diamonds** → Represent **relationships** (e.g., "Has", "Manages").
- **Lines** → Show connections between entities and relationships.
 - A **crow's foot** at one end indicates a **many** relationship (e.g., one doctor can have many appointments).

Let's See the breakdown of an ER Model of Hospital



1.6 figure ER r/ship Diagram for HMS

Entities:

- **Patient:** Stores information about patients, including patient ID, name, contact details, and address.
- **Doctor:** Contains details about doctors, such as doctor ID, name, specialty, contact information, and department.
- **Appointment:** Represents patient appointments with doctors, including attributes like appointment ID, appointment date and time, patient ID, doctor ID, and additional notes.
- **Staff:** Stores information about hospital staff members, including staff ID, name, position, and contact details.
- **Department:** Contains details about different hospital departments, each identified by a department ID and name.
- **Medical Record:** Stores medical records for patients, including patient ID, doctor ID, diagnosis, treatment, and other relevant medical information.

Relationships:

- **Patient–Appointment (Many-to-Many):** A patient can have multiple appointments, and each appointment can involve multiple patients.
- **Doctor–Appointment (One-to-Many):** A doctor can have multiple appointments, but each appointment is associated with only one doctor.
- **Doctor–Department (Many-to-One):** Many doctors can work in one department, but each doctor belongs to only one department.
- **Staff–Department (Many-to-One):** Many staff members can work in one department, but each staff member belongs to only one department.
- **Medical Record–Patient (One-to-One):** Each patient has one medical record, and each medical record is associated with only one patient.

Constraints in the Conceptual Design:

1. **Primary Key (PK)**
 - Ensures each record in an entity (table) is unique.
 - Example:
 - patient_id in the **Patient** table
 - doctor_id in the **Doctor** table
 - appointment_id in the **Appointment** table
2. **Foreign Key (FK)**
 - Establishes relationships between entities.
 - Example:
 - patient_id in the **Appointment** table (references patient_id in **Patient**)
 - doctor_id in the **Appointment** table (references doctor_id in **Doctor**)
3. **Not Null Constraint**
 - Ensures that important fields cannot have empty values.
 - Example:
 - patient_name in the **Patient** table should not be NULL
 - doctor_specialty in the **Doctor** table should not be NULL
4. **Unique Constraint**
 - Ensures no duplicate values for a specific attribute.

2.2 Logical Design

Logical Database Design

The logical database design is the next step after the conceptual design. It involves converting the **Entity-Relationship (ER)** model into a **relational model**, defining tables, columns, and data types, while still being independent of any specific database system (like MySQL , Oracle, etc.).

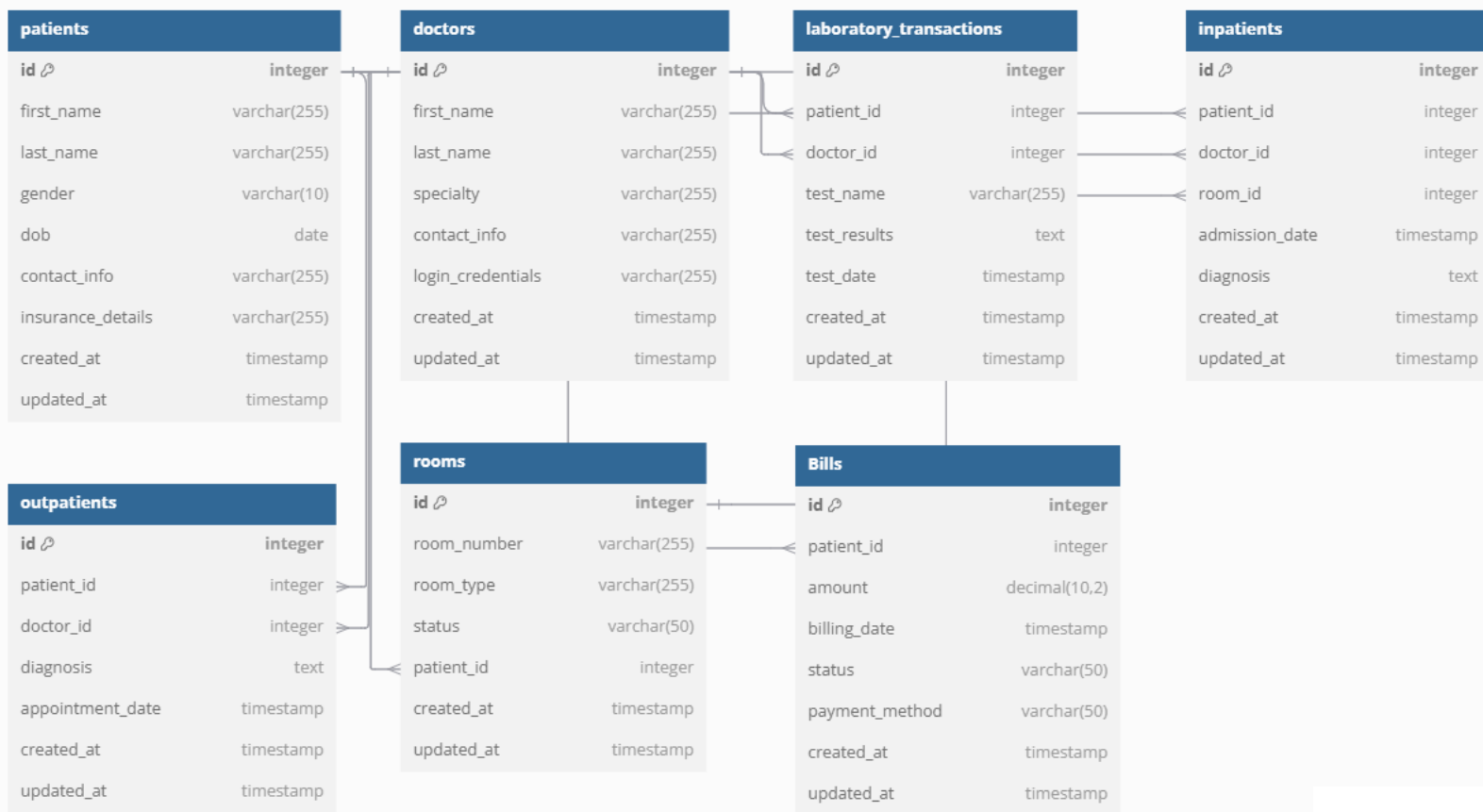
In logical design, we focus on the **structure** of the database, but without yet considering physical storage details.

Here's how the **logical design looks**.

Logical Design Diagram

The logical design can be represented visually through an **ER diagram**, where we can see the entities, their attributes, and the relationships between them.

Here is a **visual ER diagram** for this design!!



Logical design for the **Hospital Management System (HMS)**, using the specific entities provided:

1. Manage Patients Table

This table stores all the patient-related information.

1. Patients Table

Column Name	Data Type	Description
Id	Integer	Unique identifier for each patient (PK)
first_name	Varchar(255)	Patient's first name
last_name	Varchar(255)	Patient's last name
Gender	Varchar(10)	Patient's gender (Male/Female)
Dob	Date	Patient's date of birth
contact_info	Varchar(255)	Patient's contact details (phone, email, etc.)
insurance_details	Varchar(255)	Patient's insurance details
created_at	Timestamp	Record creation timestamp
updated_at	Timestamp	Record last update timestamp

This table manages basic patient details like f_name, l_name, gender, contact information, insurance details, create_at and update_at .

2. Manage Doctor Table

This table stores all the doctor-related information, including login credentials.

Doctors Table

Column Name	Data Type	Description
Id	Integer	Unique identifier for each doctor (PK)
first_name	Varchar(255)	Doctor's first name
last_name	Varchar(255)	Doctor's last name
specialty	Varchar(255)	Doctor's medical specialty (e.g., cardiology)
contact_info	Varchar(255)	Doctor's contact details (phone, email, etc.)
login_credentials	Varchar(255)	Login credentials (username/password)
created_at	Timestamp	Record creation timestamp
updated_at	Timestamp	Record last update timestamp

This table holds details about doctors such as their `full_name`, specialty, contact info, login credentials, `Create_at` and the `updated_at` they work in.

3. Manage Laboratory Table

This table is used to store information about laboratory transactions and test results.

Column Name Data Type Description

Column Name	Data Type	Description
Id	Integer	Unique identifier for each transaction (PK)
patient_id	Integer	Foreign key referencing <code>patients.id</code>
doctor_id	Integer	Foreign key referencing <code>doctors.id</code>
test_name	Varchar(255)	Name of the test performed (e.g., blood test)
test_results	Text	Results of the test
test_date	Timestamp	Date when the test was performed
created_at	Timestamp	Record creation timestamp
updated_at	Timestamp	Record last update timestamp

This table tracks laboratory tests performed for each patient, storing details like the test type, results, and date, record creation and last update.

4. Manage Inpatient Table

This table stores inpatient-related information and diagnosis.

Column Name	Data Type	Description
Id	Integer	Unique identifier for each inpatient (PK)
patient_id	Integer	Foreign key referencing <code>patients.id</code>
doctor_id	Integer	Foreign key referencing <code>doctors.id</code>
room_id	Integer	Foreign key referencing <code>rooms.id</code>
admission_date	Timestamp	Date of inpatient admission
diagnosis	Text	Diagnosis during inpatient stay
created_at	Timestamp	Record creation timestamp
updated_at	Timestamp	Record last update timestamp

This table manages inpatient records, including `room_id`, `admission_date` and discharge dates, diagnosis, and the doctor responsible for the patient's care creation and last update.

5. Manage Outpatient Table

This table stores outpatient-related information and diagnosis.

Column Name	Data Type	Description
id	Integer	Unique identifier for each outpatient (PK)
patient_id	Integer	Foreign key referencing patients.id
doctor_id	Integer	Foreign key referencing doctors.id
diagnosis	Text	Diagnosis during outpatient visit
appointment_date	Timestamp	Date of outpatient appointment
created_at	Timestamp	Record creation timestamp
updated_at	Timestamp	Record last update timestamp

This table tracks the outpatient visits, storing the diagnosis, visit date diagnosis, and associated doctor.

6. Manage Room Table

This table stores information about rooms and patient assignments.

Column Name	Data Type	Description
Id	Integer	Unique identifier for each room (PK)
room_number	Varchar(255)	Room number or name
room_type	Varchar(255)	Type of room (e.g., single, double, ICU)
Status	Varchar(50)	Room status (e.g., available, occupied)
patient_id	Integer	Foreign key referencing patients.id (assigned patient)
created_at	Timestamp	Record creation timestamp
updated_at	Timestamp	Record last update timestamp

This table manages rooms in the hospital, with each room assigned to a patient when necessary.

7. Manage Bills Table

This table stores patient billing information and transactions.

Column Name	Data Type	Description
Id	Integer	Unique identifier for each bill (PK)
patient_id	Integer	Foreign key referencing patients.id
Amount	Decimal(10,2)	Total amount of the bill
billing_date	Timestamp	Date when the bill was generated
Status	Varchar(50)	Payment status (e.g., paid, pending)
payment_method	Varchar(50)	Payment method (e.g., cash, card, insurance)
created_at	Timestamp	Record creation timestamp
updated_at	Timestamp	Record last update timestamp

This table handles patient billing details, storing information on the total amount, payment status, and payment date.

Relationships in the Logical Design

1. **Patient → Laboratory (One-to-Many)**
 - A patient can have multiple laboratory tests, but each test is associated with only one patient.
 - In the **Laboratory** table, `patient_id` is a foreign key referencing `patient_id` in the **Patient** table.
2. **Patient → Inpatient/Outpatient (One-to-Many)**
 - A patient can have multiple inpatient or outpatient visits.
 - In the **Inpatient** and **Outpatient** tables, `patient_id` is a foreign key referencing `patient_id` in the **Patient** table.
3. **Doctor → Inpatient/Outpatient (One-to-Many)**
 - A doctor can be associated with multiple inpatient or outpatient visits.
 - In the **Inpatient** and **Outpatient** tables, `doctor_id` is a foreign key referencing `doctor_id` in the **Doctor** table.
4. **Room → Patient (One-to-One)**
 - Each room is assigned to a single patient, and each patient is assigned one room at a time.
 - In the **Room** table, `patient_id` is a foreign key referencing `patient_id` in the **Patient** table.
5. **Patient → Bills (One-to-Many)**
 - A patient can have multiple bills, but each bill is associated with only one patient.
 - In the **Bills** table, `patient_id` is a foreign key referencing `patient_id` in the **Patient** table.

As we discussed, we hope you now understand the **conceptual relationships** in the **Logical Design**. Below are the key relationships and their explanations

1. **Patient → Laboratory (One-to-Many)**: A patient can have multiple tests. patient_id in the **Laboratory** table is a foreign key to **Patient**.
 2. **Patient → Inpatient/Outpatient (One-to-Many)**: A patient can have multiple visits. patient_id in the **Inpatient** and **Outpatient** tables is a foreign key to **Patient**.
 3. **Doctor → Inpatient/Outpatient (One-to-Many)**: A doctor can have multiple visits. doctor_id in the **Inpatient** and **Outpatient** tables is a foreign key to **Doctor**.
 4. **Room → Patient (One-to-One)**: Each room is assigned to one patient. patient_id in the **Room** table is a foreign key to **Patient**.
 5. **Patient → Bills (One-to-Many)**: A patient can have multiple bills. patient_id in the **Bills** table is a foreign key to **Patient**.
- ❖ The concepts outlined above and below are essentially the same, but the following explanation will make it easier for readers to understand the logical design. By reviewing the relationships between key entities like **Patient**, **Doctor**, **Appointment**, **Billing**, and **Pharmacy**, readers will gain a clearer understanding of how these entities and their attributes are interconnected. Once you grasp these relationships, you will be able to comprehend the structure and logic behind the system. This understanding will simplify the process of designing the back-end development. Let's explore how this works and **how the structure looks on page 34**.

As I told you early The **logical definition** provides a structured representation of the entities in the system and their associated attributes, forming the foundation for database design. Below is a breakdown of each **entity** and its **attributes**:

Entities and Attributes

Entity	Attributes
Patient	PatientID, Name, Age, Gender, Contact, Medical History
Doctor	DoctorID, Name, Specialty, Contact, Availability
Appointment	AppointmentID, PatientID, DoctorID, Date, Time, Status
Billing	BillID, PatientID, TotalAmount, PaymentDate, PaymentStatus
Pharmacy	MedicineID, Name, Stock, Price, ExpiryDate

Entities and Attributes

Patient

- ✓ **PatientID:** A unique identifier for each patient, typically an integer or alphanumeric code.
- ✓ **Name:** The full name of the patient, which may include first name, last name, and possibly middle names.
- ✓ **Age:** The age of the patient, usually represented as an integer.
- ✓ **Gender:** The gender of the patient, often categorized as male, female, or other.
- ✓ **Contact:** Contact information for the patient, which may include phone numbers and email addresses.
- ✓ **Medical History:** A record of past medical conditions, treatments received, allergies, etc.

Doctor

- ✓ **DoctorID:** A unique identifier for each doctor.
- ✓ **Name:** The full name of the doctor.
- ✓ **Specialty:** The area of medicine in which the doctor specializes (e.g., cardiology, pediatrics).

- ✓ **Contact:** Contact information for the doctor.
- ✓ **Availability:** Information regarding when the doctor is available for appointments (e.g., days of the week and times).

Appointment

- ✓ **AppointmentID:** A unique identifier for each appointment scheduled.
- ✓ **PatientID:** A foreign key linking to the Patient entity to identify which patient has made the appointment.
- ✓ **DoctorID:** A foreign key linking to the Doctor entity to identify which doctor will see the patient.
- ✓ **Date:** The date on which the appointment is scheduled.
- ✓ **Time:** The time at which the appointment is set to occur.
- ✓ **Status:** Indicates whether the appointment is confirmed, canceled, completed, etc.

Billing

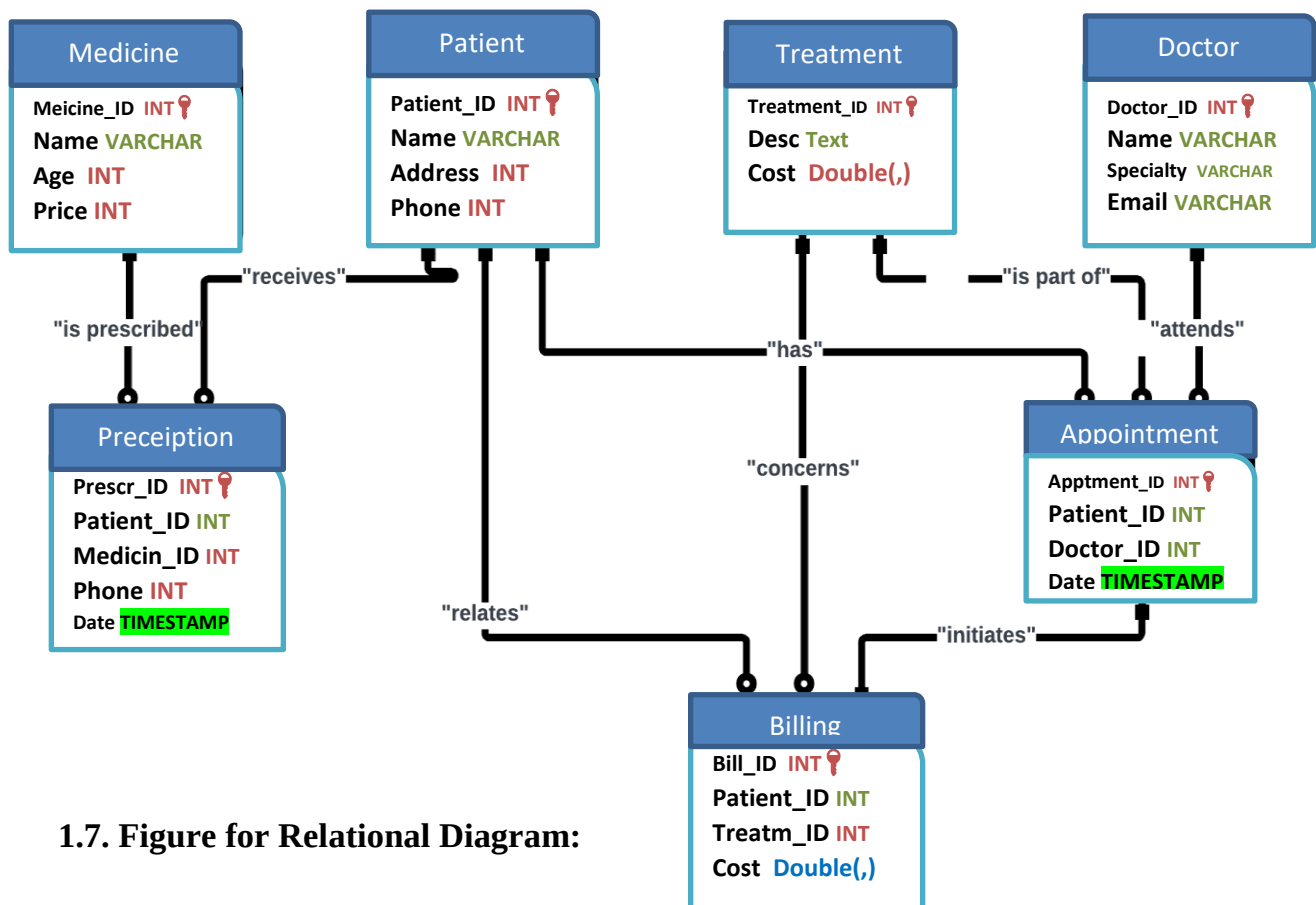
- ✓ **BillID:** A unique identifier for each billing record.
- ✓ **PatientID:** A foreign key linking to the Patient entity to associate billing with a specific patient.
- ✓ **TotalAmount:** The total amount billed for services rendered or medications provided.
- ✓ **PaymentDate:** The date on which payment was made or due.
- ✓ **PaymentStatus:** Indicates whether payment has been completed, pending, or overdue.

Pharmacy

- ✓ **MedicineID:** A unique identifier for each medicine available in stock.
- ✓ **Name:** The name of the medicine (e.g., ibuprofen).
- ✓ **Stock:** Quantity available in inventory for that particular medicine.
- ✓ **Price:** Cost per unit of medication sold at pharmacy prices.
- ✓ **ExpiryDate:** Date by which the medicine should be used before it becomes ineffective or unsafe.

Relationships

- A patient can have multiple appointments.
- Each appointment is linked to one doctor.
- Billing is associated with a patient's treatments or services.
- Prescriptions link patients to medicines in the pharmacy.



1.7. Figure for Relational Diagram:

This is a visual representation of the relationships between the tables (entities) based on ER diagram above.

You can imagine the relationships as follows:

Patient ↔ Appointment:

1. **One-to-many** relationship: One patient can have multiple appointments, but each appointment is linked to one patient.

Appointment ↔ Doctor:

1. **Many-to-one** relationship: Each appointment is associated with one doctor, but a doctor can handle multiple appointments.

Patient ↔ Billing:

1. **One-to-many** relationship: One patient can have multiple billing records, as they might undergo multiple treatments or services.

Billing ↔ Treatment:

1. **Many-to-one** relationship: Billing records are linked to treatments, with one treatment having potentially multiple billing entries.

Patient ↔ Prescription:

1. **One-to-many** relationship: A patient can have multiple prescriptions over time, but each prescription is linked to a specific patient.

Prescription ↔ Medicine:

1. **Many-to-many** relationship: A prescription can contain multiple medicines, and a medicine can be prescribed to multiple patients.

Normalization Considerations

- **1st Normal Form (1NF):** Ensure no repeating groups or arrays in the columns.
- **2nd Normal Form (2NF):** Eliminate partial dependencies (all non-key attributes must depend on the whole primary key).
- **3rd Normal Form (3NF):** Eliminate transitive dependencies (non-key attributes must depend only on the primary key).

As example we had prepared normalization part to assign students table to better understanding the normalization or normal form.

The table contains information about a team working on a project. Each row represents a team member, and it includes their **Team ID, Name, Role, Responsibilities, Phone Number, Start Date, End Date, and Project Status**.

Issues with the Original Table:

- **Redundancy:** Information like **Start Date, End Date, and Project Status** is repeated for each team member, which leads to data redundancy.
- **Normalization:** To optimize the data, we need to break it down into smaller tables to reduce redundancy and dependency, ensuring better data integrity and consistency.

Now, we will go through the normalization process step-by-step to organize the table more efficiently. We'll focus on eliminating partial dependencies, transitive dependencies, and ensuring all data is stored in a way that adheres to the rules of **1NF, 2NF, and 3NF**.

Let's walk through the normalization process for the table you provided, including **Phone, Email, Start Date, End Date, and Project Status**, step by step for each normal form

Original Table:

Team ID	Name	Role	Responsibilities	Phone	Start Date	End Date	Project Status
4493	MUT	Project Manager	Oversees project planning, progress, and delivery.	0932093288	12/12/2024	12/31/2025	Completed
4429	MEDESET	System Analyst	Gathers requirements and ensures they are feasible.	0921789012	12/12/2024	12/31/2025	Completed
3946	ABAYNESH	Database Designer	Creates the database structure and ensures data relationships.	0989214532	12/12/2024	12/31-2025	Completed
4481	MOHAMED	SQL Developer	Writes SQL queries and implements database operations.	0917238971	12/12/2024	12/31/2025	Completed
4704	ALEMU	Tester	Validates the system functionality and fixes errors.	0987345127	12/12/2024	12/31/2025	Completed
4731	YOHANS	Documentation Specialist	Prepares all required reports and documentation.	0945132891	12/12/2024	12/31/2025	Completed

Step 1: First Normal Form (1NF)

In **1NF**, the table satisfies:

- Atomic values: No multiple values per cell.
- No repeating groups: Each field contains a single value.
- Unique rows: No duplicated rows.

- Defined primary key: Each team member can be uniquely identified by **Team ID**.

Step 2: Second Normal Form (2NF)

To achieve **2NF**, we separate the **Start Date**, **End Date**, and **Project Status** (which are project-specific) into a separate table to eliminate partial dependencies.

2NF Normalized Tables:

1. **Team Member Table** (Contains **Team ID**, **Name**, **Role**, **Responsibilities**, **Phone**, **Email**):

Team ID	Name	Role	Responsibilities	Phone	Email
4493	MUT	Project Manager	Oversees project planning, progress, and delivery.	0932093288	mutgatkekdeng@gmail.com
4429	MEDESET	System Analyst	Gathers requirements and ensures they are feasible.	0921789012	Medeset87@gmail.com
3946	ABAYNESH	Database Designer	Creates the database structure and ensures data relationships.	0989214532	Abaye73@gmail.com
4481	MOHAMED	SQL Developer	Writes SQL queries and implements database operations.	0917238971	Tahir17@gmail.com
4704	ALEMU	Tester	Validates the system functionality and fixes errors.	0987345127	Alex12IT@gmail.com
4731	YOHANS	Documentation Specialist	Prepares all required reports and documentation.	0945132891	Yohanis34@gmail.com

2. **Project Table** (Contains **Start Date**, **End Date**, and **Project Status**):

Project ID	Start Date	End Date	Project Status
1	12/12/2024	12/31/2025	Completed

Step 3: Third Normal Form (3NF)

Both tables are now in **3NF** because:

- In the **Team Member Table**, there are no transitive dependencies. **Name**, **Role**, **Responsibilities**, **Phone**, and **Email** depend solely on **Team ID**.
- In the **Project Table**, **Start Date**, **End Date**, and **Project Status** are dependent only on the **Project ID**.

Final Normalized Tables (In 3NF):

1. **Team Member Table** (In 3NF):

Team ID	Name	Role	Responsibilities	Phone	Email
4493	MUT	Project Manager	Oversees project planning, progress, and delivery.	0932093288	mutgatkekdeng@gmail.com
4429	MEDESET	System Analyst	Gathers requirements and ensures they are feasible.	0921789012	Medeset87@gmail.com
3946	ABAYNESH	Database Designer	Creates the database structure and ensures data relationships.	0989214532	Abaye73@gmail.com
4481	MOHAMED	SQL Developer	Writes SQL queries and implements database operations.	0917238971	Tahir17@gmail.com
4704	ALEMU	Tester	Validates the system functionality and fixes errors.	0987345127	Alex12IT@gmail.com
4731	YOHANS	Documentation Specialist	Prepares all required reports and documentation.	0945132891	Yohanis34@gmail.com

2. **Project Table** (In 3NF):

Project ID	Start Date	End Date	Project Status
1	12/12/2024	12/31/2025	Completed

This normalization structure ensures data consistency, reduces redundancy, and organizes the project and team member information effectively.

2.3 Physical Design

1. What Is Physical Design?

Physical design takes the logical database design and implements it in a specific Relational Database Management System (RDBMS) environment. This stage focuses on performance, storage, and implementation details, including:

- **Data Types and Storage:** How each attribute is stored, including specific data type sizes and formats.
- **Indexes:** Creating indexes on frequently queried columns to speed up retrieval.
- **Partitioning:** Dividing large tables into smaller, manageable pieces.
- **Table Space Management:** Allocating storage for tables and indexes.
- **Clustering and Performance Tuning:** Adjusting physical parameters to optimize I/O performance.
- **Security Settings and Backup Strategies:** Configuring access controls, logging, and recovery options.

2. Physical Design Considerations for HMS

Data Type Specifications and Storage Settings

- **Numeric Types:** Ensure that integer types are sized appropriately (e.g., INT vs. BIGINT) based on expected volume.
- **String Types:** Use VARCHAR with a length that minimizes wasted space.
- **Timestamps and Dates:** Use TIMESTAMP or DATETIME as needed, and consider time zone handling.
- **Decimal Precision:** For financial data (e.g., billing amounts), ensure DECIMAL(10, 2) is sufficient.

Indexing

- **Primary Keys:** Indexed by default.
- **Foreign Keys:** Create indexes on columns such as patient_id, doctor_id, and room_id in related tables for faster joins.
- **Search and Filter Columns:** If queries often filter on appointment_date, billing_date, or test_date, consider indexing these columns.

Partitioning and Clustering

- **Large Tables:** If tables like laboratory_transactions or bills are expected to grow large, consider partitioning based on date (e.g., monthly partitions) or patient ranges.
- **Clustering:** Some RDBMS allow clustering of tables based on specific keys to optimize disk access.

Physical Storage and Performance Tuning

- **Tablespaces:** Allocate dedicated tablespaces for high-activity tables.
- **Buffer Pool Configuration:** Tune the buffer pool or cache settings to suit the workload.
- **I/O Optimization:** Consider storage on SSDs for faster access if the workload is high.

Backup, Recovery, and Security

- **Backup Strategies:** Plan for regular full and incremental backups.
- **Access Controls:** Enforce strict permissions at the physical level.
- **Logging:** Configure transaction logging and recovery parameters for high availability.

3. Sample Physical DDL for HMS

Below are sample SQL DDL statements (using MySQL syntax) that reflect physical design choices. Note the added index definitions and comments on potential partitioning or tablespace options.

```
CREATE DATABASE HMS;
```

Patients Table

```
CREATE TABLE patients (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  first_name VARCHAR(255) NOT NULL,  
  last_name VARCHAR(255) NOT NULL,  
  gender VARCHAR(10) NOT NULL,  
  dob DATE NOT NULL,  
  contact_info VARCHAR(255),  
  insurance_details VARCHAR(255),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP  
)
```

ENGINE=InnoDB

-- You might assign a specific tablespace, for example: **TABLESPACE** ts_patients;

```
CREATE TABLE doctors (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  first_name VARCHAR(255) NOT NULL,  
  last_name VARCHAR(255) NOT NULL,  
  specialty VARCHAR(255),  
  contact_info VARCHAR(255),  
  login_credentials VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP  
)  
ENGINE=InnoDB;
```

Laboratory Transactions Table

```
CREATE TABLE laboratory_transactions (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  patient_id INT NOT NULL,  
  doctor_id INT NOT NULL,  
  test_name VARCHAR(255),  
  test_results TEXT,  
  test_date TIMESTAMP,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP,  
  CONSTRAINT fk_lab_patient FOREIGN KEY (patient_id) REFERENCES  
  patients(id),  
  CONSTRAINT fk_lab_doctor FOREIGN KEY (doctor_id) REFERENCES doctors(id)  
)  
ENGINE=InnoDB;
```

-- Consider indexing test_date if filtering by date is common:

```
CREATE INDEX idx_test_date ON laboratory_transactions(test_date);
```

Inpatients Table

```
CREATE TABLE inpatients (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  patient_id INT NOT NULL,  
  doctor_id INT NOT NULL,  
  room_id INT NOT NULL,  
  admission_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  diagnosis TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP,  
  CONSTRAINT fk_inpatient_patient FOREIGN KEY (patient_id) REFERENCES  
  patients(id),  
  CONSTRAINT fk_inpatient_doctor FOREIGN KEY (doctor_id) REFERENCES  
  doctors(id),  
  CONSTRAINT fk_inpatient_room FOREIGN KEY (room_id) REFERENCES  
  rooms(id)  
)  
ENGINE=InnoDB;  
-- Indexing foreign keys:  
CREATE INDEX idx_inpatient_patient ON inpatients(patient_id);  
CREATE INDEX idx_inpatient_doctor ON inpatients(doctor_id);
```

Outpatients Table

```
CREATE TABLE outpatients (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  patient_id INT NOT NULL,  
  doctor_id INT NOT NULL,  
  diagnosis TEXT,  
  appointment_date TIMESTAMP,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP,  
  CONSTRAINT fk_outpatient_patient FOREIGN KEY (patient_id) REFERENCES  
  patients(id),  
  CONSTRAINT fk_outpatient_doctor FOREIGN KEY (doctor_id) REFERENCES  
  doctors(id)  
)  
ENGINE=InnoDB;
```

```
CREATE INDEX idx_appointment_date ON outpatients(appointment_date);
```

Rooms Table

```
CREATE TABLE rooms (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  room_number VARCHAR(255) NOT NULL UNIQUE,  
  room_type VARCHAR(255),  
  status VARCHAR(50),  
  patient_id INT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP,  
  CONSTRAINT fk_room_patient FOREIGN KEY (patient_id) REFERENCES  
  patients(id)  
)  
ENGINE=InnoDB;  
-- Depending on usage, partition by room_type may be considered if there are many room  
entries.
```

Bills Table

```
CREATE TABLE bills (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  patient_id INT NOT NULL,  
  amount DECIMAL(10,2) NOT NULL,  
  billing_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  status VARCHAR(50) NOT NULL,  
  payment_method VARCHAR(50),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP,  
  CONSTRAINT fk_bill_patient FOREIGN KEY (patient_id) REFERENCES patients(id)  
)  
ENGINE=InnoDB;  
CREATE INDEX idx_billing_date ON bills(billing_date);
```

4. Implementation Considerations

Performance Tuning and Scalability

- **Index Optimization:** Analyze query patterns; add composite indexes if queries filter on multiple columns.
- **Partitioning:** For very large tables like bills or laboratory_transactions, partition the table by billing_date or test_date respectively.
- **Caching:** Use query caching provided by the RDBMS to speed up frequent reads.

Security and Maintenance

- **Data Encryption:** Encrypt sensitive data (e.g., login credentials) using database-supported encryption functions.
- **Regular Backups:** Schedule regular full and incremental backups.
- **Monitoring and Logging:** Implement logging for queries and monitor performance using database-specific tools.

Hardware and Environment

- **SSD Storage:** Consider SSDs for faster I/O performance.
- **Memory Allocation:** Adjust buffer pool size and cache settings in your RDBMS configuration.
- **Load Balancing:** For high-traffic systems, consider replication and clustering.

Here is the **physical design** we will implement for the following entities: Patient, Doctor, Appointment, Billing, and Pharmacy.

As **physical design** involves the implementation of the database structure at the physical level. This includes creating SQL tables, indexes, and storage structures that align with the logical design and the requirements of the application. **The physical design** focuses on how data is stored, managed, and accessed efficiently.

Database Schema

SQL tables are designed to store and manage all the necessary data.

SQL Table for Patients

```
CREATE TABLE Patients (  
    PatientID INT AUTO_INCREMENT PRIMARY KEY,  
    Name VARCHAR(100),  
    Age INT,  
    Gender VARCHAR(10),  
    Contact VARCHAR(50),  
    MedicalHistory TEXT  
);
```

SQL Table for Doctors

```
CREATE TABLE Doctors (  
    DoctorID INT AUTO_INCREMENT PRIMARY KEY,  
    Name VARCHAR(100),  
    Specialty VARCHAR(50),  
    Contact VARCHAR(50),  
    Availability TEXT  
);
```

SQL Table for Appointments

```
CREATE TABLE Appointments (  
    AppointmentID INT AUTO_INCREMENT PRIMARY KEY,  
    PatientID INT,  
    DoctorID INT,  
    Date DATE,  
    Time TIME,  
    Status VARCHAR(20),  
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),  
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)  
);
```

SQL Table for Billing

```
CREATE TABLE Billing (  
    BillID INT AUTO_INCREMENT PRIMARY KEY,  
    PatientID INT,  
    TotalAmount DECIMAL(10, 2),  
    PaymentDate DATE,  
    PaymentStatus VARCHAR(50),  
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)  
);
```

SQL Table for Pharmacy

```
CREATE TABLE Pharmacy (  
    MedicineID INT AUTO_INCREMENT PRIMARY KEY,  
    Name VARCHAR(100),  
    Stock INT,  
    Price DECIMAL(10, 2),  
    ExpiryDate DATE  
);
```

Phase 3: Implementation Phase

Phase refers to the stage in a project where the design is brought to life through coding, configuration, and integration of components. For a **database project**, this phase focuses on [setting up](#) the database and ensuring it contains necessary data for operation.

Implementing Hospital Management System | Environment Creation:

Required Softwares:

- Xampp software (for PHP and MySQL)
- VS Code (you can use any other suitable editor as well)
- Install Bootstrap or download Bootstrap extension on VS Code.

After we downloaded the above required software, we will start creating the project. In the following article, we will discuss different modules compiled within the same category.

We will discuss it stepwise:

Implementing Hospital Management System | Database Creation:

Go to your favorite browser and write `localhost/dashboard >> phpmyadmin`.

Now by followed that we can create our own database by using the **New** button.

Database Used in this project:

Below is the SQL code to create those tables in the database. You can modify the code to create your own database for the project.

Implementation Considerations

1. Data Types and Storage:

- Choose appropriate data types (e.g., INT for IDs, VARCHAR(255) for names) to optimize storage.
- Use TIMESTAMP for creation/update tracking and DECIMAL for monetary values.

2. Indexes:

- Primary keys are automatically indexed.
- Create additional indexes on frequently queried columns (e.g., foreign keys such as `patient_id`, `doctor_id`, and date fields).

3. Foreign Keys and Referential Integrity:

- Enforce relationships with foreign keys to ensure data consistency (e.g., linking laboratory transactions to a valid patient and doctor).

4. Partitioning and Tablespaces:

- For high-volume tables (e.g., bills or laboratory_transactions), consider partitioning by date.
- Assign high-activity tables to dedicated tablespaces if your RDBMS supports it.

5. Performance Tuning and Scalability:

- Monitor query performance and adjust indexing strategies as needed.
- Consider replication, clustering, or sharding for scalability.

6. Security:

- Ensure that sensitive fields (like `login_credentials`) are stored securely (using hashing or encryption).
- Set up proper access controls and roles in the database system.

7. Backup and Recovery:

- Implement a robust backup schedule and recovery plan to minimize data loss.

Sample DDL and Data Insertion

Below are sample SQL DDL statements (using SQL syntax) for each table, followed by multiple INSERT statements that add distinct records.

"As we all know, the creation process cannot be executed without knowing the database name. Therefore, let us create a database named 'HMS'."

To create a database for a Hospital Management System (HMS) in SQL, the syntax would be:

```
CREATE DATABASE HMS ;
```

To implement this **team members table** using SQL, we had created a table with the relevant columns (Team ID, Name, Role, Responsibilities, Phone, Email), and we had already insert the data for each team member. Here's how we had done the :

SQL Implementation

1. Create the Team Members Table:

DDL:

```
CREATE TABLE team_members (  
    team_id INT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    role VARCHAR(255) NOT NULL,  
    responsibilities TEXT,  
    phone INT(15),  
    email VARCHAR(255)  
);
```

This SQL command creates the `team_members` table with the following columns:

- **team_id**: A unique identifier for each team member (Primary Key).
- **name**: The name of the team member.
- **role**: The role of the team member.
- **responsibilities**: Describes the responsibilities of the team member.
- **phone**: The contact number of the team member.
- **email**: The email address of the team member.

2. Insert Data into the Team Members Table:

```
INSERT INTO team_members (team_id, name, role, responsibilities, phone, email)
VALUES
('4493', 'MUT', 'Project Manager', 'Oversees project planning, progress, and
delivery.', '0932093288', 'mutgatkekdeng@gmail.com'),
('4429', 'MEDESET', 'System Analyst', 'Gathers requirements and ensures they
are feasible.', '0921789012', 'Medeset87@gmail.com'),
('3946', 'ABAYNESH', 'Database Designer', 'Creates the database structure and
ensures data relationships.', '0989214532', 'Abaye73@gmail.com'),
('4481', 'MOHAMED', 'SQL Developer', 'Writes SQL queries and implements
database operations.', '0917238971', 'Tahir17@gmail.com'),
('4704', 'ALEMU', 'Tester', 'Validates the system functionality and fixes
errors.', '0987345127', 'Alex12IT@gmail.com'),
('4731', 'YOHANS', 'Documentation Specialist', 'Prepares all required reports
and documentation.', '0945132891', 'Yohanis34@gmail.com');
```

This SQL command inserts the provided team members' data into the `team_members` table.

3. Query to View the Team Members Table:

To view the data you've inserted, you can use the following query:

```
SELECT * FROM team_members;
```

Explanation:

- The `CREATE TABLE` statement defines the structure of the `team_members` table with columns for `team_id`, `name`, `role`, `responsibilities`, `phone`, and `email`.
- The `INSERT INTO` statement adds the data for each team member into the table.
- The `SELECT * FROM team_members;` command retrieves all the data from the table to verify that it has been inserted correctly.

Optional Enhancements:

1. **Data Integrity:** You can add **NOT NULL** constraints for the columns like `name`, `role`, and `email` to ensure they must contain values.
2. **Unique Constraints:** You could enforce uniqueness on the `email` column to prevent duplicate email addresses:
3. `ALTER TABLE team_members ADD CONSTRAINT unique_email UNIQUE (email);`

1. Patients Table

DDL:

```
CREATE TABLE patients (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    first_name VARCHAR(255) NOT NULL,  
    last_name VARCHAR(255) NOT NULL,  
    gender VARCHAR(10) NOT NULL,  
    dob DATE NOT NULL,  
    contact_info VARCHAR(255),  
    insurance_details VARCHAR(255),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);  
ENGINE=InnoDB;
```

Sample INSERTs:

```
INSERT INTO patients (first_name, last_name, gender, dob, contact_info,  
insurance_details)  
VALUES ('Alemu', 'Tadesse', 'Male', '1985-06-15', 'alemu.t@example.com', 'Gold  
Health');
```

```
INSERT INTO patients (first_name, last_name, gender, dob, contact_info,  
insurance_details)  
VALUES ('Birtukan', 'Assefa', 'Female', '1990-03-22',  
'birtu.assefa@example.com', 'Platinum Care');
```

```
INSERT INTO patients (first_name, last_name, gender, dob, contact_info,  
insurance_details)  
VALUES ('Caleb', 'Mulugeta', 'Male', '1988-11-10', 'caleb.m@example.com',  
'Standard Health');
```

```
INSERT INTO patients (first_name, last_name, gender, dob, contact_info,  
insurance_details)  
VALUES ('Daniela', 'Tesfaye', 'Female', '1992-07-05', 'daniela.t@example.com',  
'Basic Health');
```

```
INSERT INTO patients (first_name, last_name, gender, dob, contact_info,  
insurance_details)  
VALUES ('Eyerus', 'Berhanu', 'Female', '1987-01-30', 'eyerus.b@example.com',  
'Gold Health');
```

```
INSERT INTO patients (first_name, last_name, gender, dob, contact_info,  
insurance_details)  
VALUES ('Fikadu', 'Alemu', 'Male', '1991-12-12', 'fikadu.a@example.com',  
'Standard Health');
```

2. Doctors Table

DDL:

```

CREATE TABLE doctors (
    id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(255) NOT NULL,
    last_name VARCHAR(255) NOT NULL,
    specialty VARCHAR(255),
    contact_info VARCHAR(255),
    login_credentials VARCHAR(255) NOT NULL, -- Ensure hashed storage
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
)
ENGINE=InnoDB;

```

Sample INSERTs:

```

INSERT INTO doctors (first_name, last_name, specialty, contact_info,
login_credentials)
VALUES ('Mekdes', 'Abebe', 'Cardiology', 'mekdes.abebe@hospital.com',
'hashed_pw1');
INSERT INTO doctors (first_name, last_name, specialty, contact_info,
login_credentials)
VALUES ('Yohannes', 'Desta', 'Neurology', 'yohannes.d@hospital.com',
'hashed_pw2');
INSERT INTO doctors (first_name, last_name, specialty, contact_info,
login_credentials)
VALUES ('Selamawit', 'Tesfaye', 'Pediatrics', 'selamawit.t@hospital.com',
'hashed_pw3');

INSERT INTO doctors (first_name, last_name, specialty, contact_info,
login_credentials)
VALUES ('Girma', 'Tadesse', 'Orthopedics', 'girma.t@hospital.com',
'hashed_pw4');

INSERT INTO doctors (first_name, last_name, specialty, contact_info,
login_credentials)
VALUES ('Aster', 'Gebremedhin', 'Dermatology', 'aster.g@hospital.com',
'hashed_pw5');

INSERT INTO doctors (first_name, last_name, specialty, contact_info,
login_credentials)
VALUES ('Samuel', 'Kassahun', 'General Surgery', 'samuel.k@hospital.com',
'hashed_pw6');

```

3. Laboratory Transactions Table

DDL:

```

CREATE TABLE laboratory_transactions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    test_name VARCHAR(255),
    test_results TEXT,
    test_date TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

```

```

        CONSTRAINT fk_lab_patient FOREIGN KEY (patient_id) REFERENCES patients(id),
        CONSTRAINT fk_lab_doctor FOREIGN KEY (doctor_id) REFERENCES doctors(id)
    )
ENGINE=InnoDB;
CREATE INDEX idx_test_date ON laboratory_transactions(test_date);

```

Sample INSERTs:

```

INSERT INTO laboratory_transactions (patient_id, doctor_id, test_name,
test_results, test_date)
VALUES (1, 2, 'Blood Test', 'Normal hemoglobin levels', '2023-03-15 09:00:00');

INSERT INTO laboratory_transactions (patient_id, doctor_id, test_name,
test_results, test_date)
VALUES (2, 3, 'X-Ray', 'No fractures detected', '2023-03-16 10:30:00');

INSERT INTO laboratory_transactions (patient_id, doctor_id, test_name,
test_results, test_date)
VALUES (3, 1, 'MRI Scan', 'Minor anomalies observed', '2023-03-17 14:00:00');

INSERT INTO laboratory_transactions (patient_id, doctor_id, test_name,
test_results, test_date)
VALUES (4, 4, 'CT Scan', 'Clear scan', '2023-03-18 11:15:00');

INSERT INTO laboratory_transactions (patient_id, doctor_id, test_name,
test_results, test_date)
VALUES (5, 5, 'Urine Test', 'Slight infection indicated', '2023-03-19
08:45:00');

INSERT INTO laboratory_transactions (patient_id, doctor_id, test_name,
test_results, test_date)
VALUES (6, 6, 'Electrocardiogram', 'Normal rhythm', '2023-03-20 13:30:00');

```

4. Inpatients Table

DDL:

```

CREATE TABLE inpatients (
    id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    room_id INT NOT NULL,
    admission_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    diagnosis TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

```

```

        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
        CONSTRAINT fk_inpatient_patient FOREIGN KEY (patient_id) REFERENCES
patients(id),
        CONSTRAINT fk_inpatient_doctor FOREIGN KEY (doctor_id) REFERENCES
doctors(id),
        CONSTRAINT fk_inpatient_room FOREIGN KEY (room_id) REFERENCES rooms(id)
    )
ENGINE=InnoDB;
CREATE INDEX idx_inpatient_patient ON inpatients(patient_id);
CREATE INDEX idx_inpatient_doctor ON inpatients(doctor_id);

```

Sample INSERTs:

```

INSERT INTO inpatients (patient_id, doctor_id, room_id, admission_date,
diagnosis)
VALUES (1, 1, 1, '2023-03-10 08:00:00', 'Acute Appendicitis');

```

```

INSERT INTO inpatients (patient_id, doctor_id, room_id, admission_date,
diagnosis)
VALUES (2, 2, 2, '2023-03-11 09:30:00', 'Pneumonia');

```

```

INSERT INTO inpatients (patient_id, doctor_id, room_id, admission_date,
diagnosis)
VALUES (3, 3, 3, '2023-03-12 07:45:00', 'Fractured Femur');

```

```

INSERT INTO inpatients (patient_id, doctor_id, room_id, admission_date,
diagnosis)
VALUES (4, 4, 4, '2023-03-13 10:00:00', 'Severe Dehydration');

```

```

INSERT INTO inpatients (patient_id, doctor_id, room_id, admission_date,
diagnosis)
VALUES (5, 5, 5, '2023-03-14 11:20:00', 'Acute Pancreatitis');

```

```

INSERT INTO inpatients (patient_id, doctor_id, room_id, admission_date,
diagnosis)
VALUES (6, 6, 6, '2023-03-15 12:00:00', 'Gallstones');

```

5. Outpatients Table

DDL:

```
CREATE TABLE outpatients (
    id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    diagnosis TEXT,
    appointment_date TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_outpatient_patient FOREIGN KEY (patient_id) REFERENCES
patients(id),
    CONSTRAINT fk_outpatient_doctor FOREIGN KEY (doctor_id) REFERENCES
doctors(id)
)
ENGINE=InnoDB;
CREATE INDEX idx_appointment_date ON outpatients(appointment_date);
```

Sample INSERTs:

```
INSERT INTO outpatients (patient_id, doctor_id, diagnosis, appointment_date)
VALUES (1, 2, 'Seasonal Allergies', '2023-03-21 09:00:00');
INSERT INTO outpatients (patient_id, doctor_id, diagnosis, appointment_date)
VALUES (2, 3, 'Flu Symptoms', '2023-03-22 10:15:00');
```

```
INSERT INTO outpatients (patient_id, doctor_id, diagnosis, appointment_date)
VALUES (3, 1, 'Migraine', '2023-03-23 11:30:00');
```

```
INSERT INTO outpatients (patient_id, doctor_id, diagnosis, appointment_date)
VALUES (4, 4, 'Back Pain', '2023-03-24 14:00:00');
```

```
INSERT INTO outpatients (patient_id, doctor_id, diagnosis, appointment_date)
VALUES (5, 5, 'Skin Rash', '2023-03-25 08:45:00');
```

```
INSERT INTO outpatients (patient_id, doctor_id, diagnosis, appointment_date)
VALUES (6, 6, 'Digestive Issues', '2023-03-26 13:15:00');
```

6. Rooms Table

DDL:

```
CREATE TABLE rooms (
    id INT AUTO_INCREMENT PRIMARY KEY,
    room_number VARCHAR(255) NOT NULL UNIQUE,
    room_type VARCHAR(255),
    status VARCHAR(50),
    patient_id INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_room_patient FOREIGN KEY (patient_id) REFERENCES patients(id)
)
```

```
ENGINE=InnoDB;
```

Sample INSERTs:

```
INSERT INTO rooms (room_number, room_type, status, patient_id)
VALUES ('101A', 'ICU', 'Occupied', 1);
```

```
INSERT INTO rooms (room_number, room_type, status, patient_id)
VALUES ('102B', 'Single', 'Occupied', 2);
```

```
INSERT INTO rooms (room_number, room_type, status, patient_id)
VALUES ('103C', 'Double', 'Occupied', 3);
```

```
INSERT INTO rooms (room_number, room_type, status, patient_id)
VALUES ('104D', 'Single', 'Occupied', 4);
```

```
INSERT INTO rooms (room_number, room_type, status, patient_id)
VALUES ('105E', 'ICU', 'Occupied', 5);
```

```
INSERT INTO rooms (room_number, room_type, status, patient_id)
VALUES ('106F', 'Double', 'Occupied', 6);
```

7. Bills Table

DDL:

```
CREATE TABLE bills (
    id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    billing_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(50) NOT NULL,
    payment_method VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_bill_patient FOREIGN KEY (patient_id) REFERENCES patients(id)
)
ENGINE=InnoDB;
CREATE INDEX idx_billing_date ON bills(billing_date);
```

Sample INSERTs:

```
INSERT INTO bills (patient_id, amount, billing_date, status, payment_method)
VALUES (1, 500.00, '2023-03-27 09:00:00', 'Pending', 'Cash');
```

```
INSERT INTO bills (patient_id, amount, billing_date, status, payment_method)
VALUES (2, 750.00, '2023-03-27 10:15:00', 'Paid', 'Card');
```

```
INSERT INTO bills (patient_id, amount, billing_date, status, payment_method)
```

```
VALUES (3, 620.50, '2023-03-27 11:30:00', 'Pending', 'Insurance');

INSERT INTO bills (patient_id, amount, billing_date, status, payment_method)
VALUES (4, 430.00, '2023-03-27 12:45:00', 'Paid', 'Cash');

INSERT INTO bills (patient_id, amount, billing_date, status, payment_method)
VALUES (5, 800.00, '2023-03-27 13:30:00', 'Pending', 'Card');

INSERT INTO bills (patient_id, amount, billing_date, status, payment_method)
VALUES (6, 950.00, '2023-03-27 14:00:00', 'Paid', 'Insurance');
```

This alternative physical design approach covers key implementation considerations—from data types, indexing, and partitioning to security and backup strategies. Additionally, the sample INSERT statements demonstrate how to populate each table with more than five distinct records. This complete design is ready for implementation in your RDBMS, ensuring data integrity, performance, and scalability for the Hospital Management System.

SQL tables are designed to store and manage all the necessary data.

SQL Table for Patients

```
CREATE TABLE Patients (
    PatientID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100),
    Age INT,
    Gender VARCHAR(10),
    Contact VARCHAR(50),
    MedicalHistory TEXT
);
```

SQL Table for Doctors

```
CREATE TABLE Doctors (
    DoctorID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100),
    Specialty VARCHAR(50),
    Contact VARCHAR(50),
    Availability TEXT
);
```

SQL Table for Appointments

```
CREATE TABLE Appointments (  
    AppointmentID INT AUTO_INCREMENT PRIMARY KEY,  
    PatientID INT,  
    DoctorID INT,  
    Date DATE,  
    Time TIME,  
    Status VARCHAR(20),  
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),  
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)  
);
```

SQL Table for Billing

```
CREATE TABLE Billing (  
    BillID INT AUTO_INCREMENT PRIMARY KEY,  
    PatientID INT,  
    TotalAmount DECIMAL(10, 2),  
    PaymentDate DATE,  
    PaymentStatus VARCHAR(50),  
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)  
);
```

SQL Table for Pharmacy

```
CREATE TABLE Pharmacy (  
    MedicineID INT AUTO_INCREMENT PRIMARY KEY,  
    Name VARCHAR(100),  
    Stock INT,  
    Price DECIMAL(10, 2),  
    ExpiryDate DATE  
);
```

To insert values into the tables you've created for **Patients, Doctors, Appointments, Billing, and Pharmacy**, here's how you can structure the `INSERT INTO` SQL statements:

1. Insert Values into the Patients Table:

```
INSERT INTO Patients (Name, Age, Gender, Contact, MedicalHistory)
VALUES
('John Doe', 30, 'Male', '1234567890', 'No known allergies'),
('Jane Smith', 45, 'Female', '0987654321', 'Hypertension, Asthma'),
('Samuel Johnson', 60, 'Male', '1122334455', 'Diabetes, High cholesterol');
```

2. Insert Values into the Doctors Table:

```
INSERT INTO Doctors (Name, Specialty, Contact, Availability)
VALUES
('Dr. Alice Green', 'Cardiologist', '5551234567', 'Mon-Fri, 9am-5pm'),
('Dr. Bob White', 'Pediatrician', '5552345678', 'Mon-Wed, 10am-3pm'),
('Dr. Charlie Brown', 'Orthopedic Surgeon', '5553456789', 'Tue-Thu, 8am-4pm');
```

3. Insert Values into the Appointments Table:

```
INSERT INTO Appointments (PatientID, DoctorID, Date, Time, Status)
VALUES
(1, 1, '2025-02-10', '10:00:00', 'Scheduled'),
(2, 2, '2025-02-11', '11:30:00', 'Scheduled'),
(3, 3, '2025-02-12', '14:00:00', 'Completed');
```

4. Insert Values into the Billing Table:

```
INSERT INTO Billing (PatientID, TotalAmount, PaymentDate, PaymentStatus)
VALUES
(1, 200.00, '2025-02-10', 'Paid'),
(2, 150.00, '2025-02-11', 'Pending'),
(3, 300.00, '2025-02-12', 'Paid');
```

5. Insert Values into the Pharmacy Table:

```
INSERT INTO Pharmacy (Name, Stock, Price, ExpiryDate)
VALUES
('Paracetamol', 100, 10.50, '2026-06-30'),
('Amoxicillin', 200, 25.00, '2025-12-31'),
('Ibuprofen', 50, 15.75, '2026-03-15');
```

Explanation:

- **Patients Table:** Inserted 3 example patients with different details.
- **Doctors Table:** Inserted 3 example doctors with different specialties and contact details.
- **Appointments Table:** Inserted 3 example appointments, associating patients with doctors, and providing date, time, and status.
- **Billing Table:** Inserted 3 billing entries for the patients with the total amount, payment date, and payment status.
- **Pharmacy Table:** Inserted 3 example medicines available in the pharmacy with stock, price, and expiry date.

To Verify the Data:

You can run a SELECT query to verify the inserted data:

Certainly! Here is the SQL code to retrieve all records (`SELECT *`) from all the tables in one block:

```
-- View Team Members Table
SELECT * FROM team_members;

-- View Patients Table
SELECT * FROM patients;

-- View Doctors Table
SELECT * FROM doctors;

-- View Laboratory Transactions Table
SELECT * FROM laboratory_transactions;

-- View Inpatients Table
SELECT * FROM inpatients;

-- View Outpatients Table
SELECT * FROM outpatients;

-- View Rooms Table
SELECT * FROM rooms;

-- View Bills Table
SELECT * FROM bills;
```

This code runs multiple `SELECT *` statements one after another, each for a different table. You can execute this block in your SQL client or query editor to retrieve all the data from these tables in a single query execution.

```
SELECT * FROM Patients;
SELECT * FROM Doctors;
SELECT * FROM Appointments;
SELECT * FROM Billing;
SELECT * FROM Pharmacy;
```

3. Implementation Considerations

Indexing and Performance

- **Primary Keys** are indexed by default.
- Create **indexes** on foreign key columns (e.g., patient_id, doctor_id in dependent tables) to speed up join operations.
- If queries frequently filter by date, consider indexing the created_at or appointment_date fields.

Security and Data Integrity

- For the **Doctors** table, store **login_credentials** securely by hashing passwords.
- Use **triggers** or **stored procedures** to enforce business rules. For instance, when an inpatient is discharged, update the room's status to “available.”
- Ensure **foreign key constraints** are enforced to maintain referential integrity between tables.

Scalability

- If expecting high volumes of data (especially in the **laboratory_transactions** and **bills** tables), consider partitioning these tables by date or patient.
- Use replication and clustering features of your chosen RDBMS to improve performance and availability.

Backup and Recovery

- Plan for regular backups and implement recovery strategies.
- Consider using transaction logs and point-in-time recovery to safeguard against data loss.

Testing and Optimization

- Once implemented, use sample data to test join operations, query performance, and the behavior of constraints.
- Regularly monitor query performance and adjust indexes or refactor queries as needed.

Conclusion:

In conclusion, the development of the Hospital Management System (HMS) followed a structured approach through its phases: Planning, Analysis, Design, and Implementation.

Planning and Analysis Phase: In this phase, our group established effective teamwork and clearly defined the scope and objectives of the system. We conducted a thorough analysis of the hospital's operations and requirements to ensure that the system would meet the needs of both the staff and patients. We identified key functionalities such as patient registration, appointment scheduling, billing, and inventory management.

Design Phase: During the design phase, we focused on three key aspects—conceptual, logical, and physical design. We conceptualized the overall architecture of the system, outlining its core features and flow. The logical design addressed how data would be managed and processed, ensuring that the system would be efficient and user-friendly. In the physical design, we mapped out the technical infrastructure and database management to support the system's operations.

Implementation Phase: In this final phase, we moved forward with actual system development, coding, and integration. The system was tested for functionality, usability, and performance. We ensured that it met the specified requirements and successfully delivered the desired results.

Through these phases, we have demonstrated the importance of proper planning, a detailed analysis of user needs, and the careful design and implementation of a system that can streamline hospital operations, improving efficiency and patient care.

REFERENCE:

Raghu Ramakrishnan and Johannes Gehrke (2003) – *Database Management Systems*

Joe Celko (1999) – *SQL For Smarties: Advanced SQL Programming*

Alan Dennis, Barbara Haley Wixom, and Roberta M. Roth (2014) – *Systems Analysis and Design*

Joseph K. S. H. Lee and Janet L. Hart (2005) – *Healthcare Information Systems: A Practical Approach for Health Care Management*

Roger S. Pressman (2014) – *Software Engineering: A Practitioner's Approach*

Michael J. Hernandez (2013) – *Database Design for Mere Mortals: A Hands-On Guide to Relational Database Design*

Anthony DeBarros (2019) – *Practical SQL: A Beginner's Guide to Storytelling with Data*

Clare Churcher (2013) – *Beginning Database Design*

FOR DRAW ER DIAGRAM VISIT THIS LINK [Untitled Diagram - draw.io](#) [Untitled Diagram - draw.io](#)